# PBL03 — Arquitetura do Processador RISC-V (ISA Limitado)

Oct 1, 2026 · @Marina Alencar

## Introdução

O PBL03 pede o desenvolvimento e a verificação, em SystemVerilog com UVM, de um processador RISC-V com um conjunto de instruções (ISA) limitado — ou seja, restrito ao mínimo necessário para executar um programa específico, em vez de implementar o RV32I completo. A disciplina organiza o trabalho em duas frentes, feitas por duplas de equipes: cada equipe projeta o seu próprio núcleo (RTL) e, ao mesmo tempo, verifica o núcleo da equipe parceira, usando o template oficial `verif-env-template` e o fluxo Vivado (xvlog/xelab/xsim). O entregável final é um relatório/apresentação com demonstração de 10 minutos.

Este documento cobre a **Frente 1 — projeto do núcleo RTL**, construído do zero a partir da microarquitetura de ciclo único apresentada no Capítulo 7 de Harris & Harris, *Digital Design and Computer Architecture* (RISC-V Edition), referência adotada pela disciplina.

O requisito que define o escopo do ISA limitado é o programa já desenvolvido no **PBL02**: o jogo Genius, escrito em assembly RISC-V. Em vez de implementar todo o RV32I, o processador aqui documentado suporta exatamente as instruções que esse programa usa (mais algumas de suporte, como `ecall`, para observação em simulação). Isso faz da Frente 1 uma continuação direta do PBL02 — o núcleo projetado aqui é, por construção, capaz de rodar o mesmo binário validado naquela etapa.

## Escopo do ISA limitado

O núcleo implementa apenas as instruções RV32I necessárias para rodar o programa do PBL02 (jogo Genius) mais `ecall`, usado como marcador de fim/observação em simulação. Nenhuma instrução fora dessa lista é decodificada corretamente.

| Instrução | Tipo | Opcode | Função |
|---|---|---|---|
| `add` | R | 0110011 | soma registradores |
| `sub` | R | 0110011 | subtração |
| `and` | R | 0110011 | AND bit a bit |
| `or` | R | 0110011 | OR bit a bit |

| Instrução | Tipo | Opcode | Função |
|---|---|---|---|
| `xor` | R | 0110011 | XOR bit a bit |
| `addi` | I | 0010011 | soma com imediato |
| `andi` | I | 0010011 | AND com imediato |
| `slli` | I | 0010011 | deslocamento lógico à esquerda |
| `srli` | I | 0010011 | deslocamento lógico à direita |
| `lw` | I | 0000011 | carrega palavra (32 bits) |
| `lb` | I | 0000011 | carrega byte (sign-extend) |
| `sw` | S | 0100011 | armazena palavra |
| `sb` | S | 0100011 | armazena byte |
| `beq` | B | 1100011 | desvia se igual |
| `bne` | B | 1100011 | desvia se diferente |
| `bge` | B | 1100011 | desvia se maior/igual (signed) |
| `bltu` | B | 1100011 | desvia se menor (unsigned) |
| `jal` | J | 1101111 | salto incondicional com retorno |
| `jalr` | I | 1100111 | salto indireto com retorno |
| `lui` | U | 0110111 | carrega imediato nos bits altos |
| `auipc` | U | 0010111 | PC + imediato nos bits altos |
| `ecall` | I | 1110011 | chamada de sistema (usada como marcador de fim) |

**Justificativa:** o programa do PBL02 usa aritmética e lógica básica para pontuação e estado do jogo ( `add` / `sub` / `and` / `or` / `xor` / `addi` / `andi` ), acesso a vetores/strings de caracteres em memória byte a byte ( `lb` / `sb` , além de `lw` / `sw` para palavras), controle de fluxo (loops e condicionais via `beq` / `bne` / `bge` / `bltu` , chamadas de sub-rotina via `jal` / `jalr` ), e carregamento de constantes/endereços grandes ( `lui` / `auipc` , necessários porque o imediato de 12 bits de `addi` não alcança os endereços de memória usados pelo jogo). `ecall` não faz parte do programa em si, mas foi adicionado como ponto de parada controlável para a validação em simulação (ver Seção 7).

# Escolha arquitetural

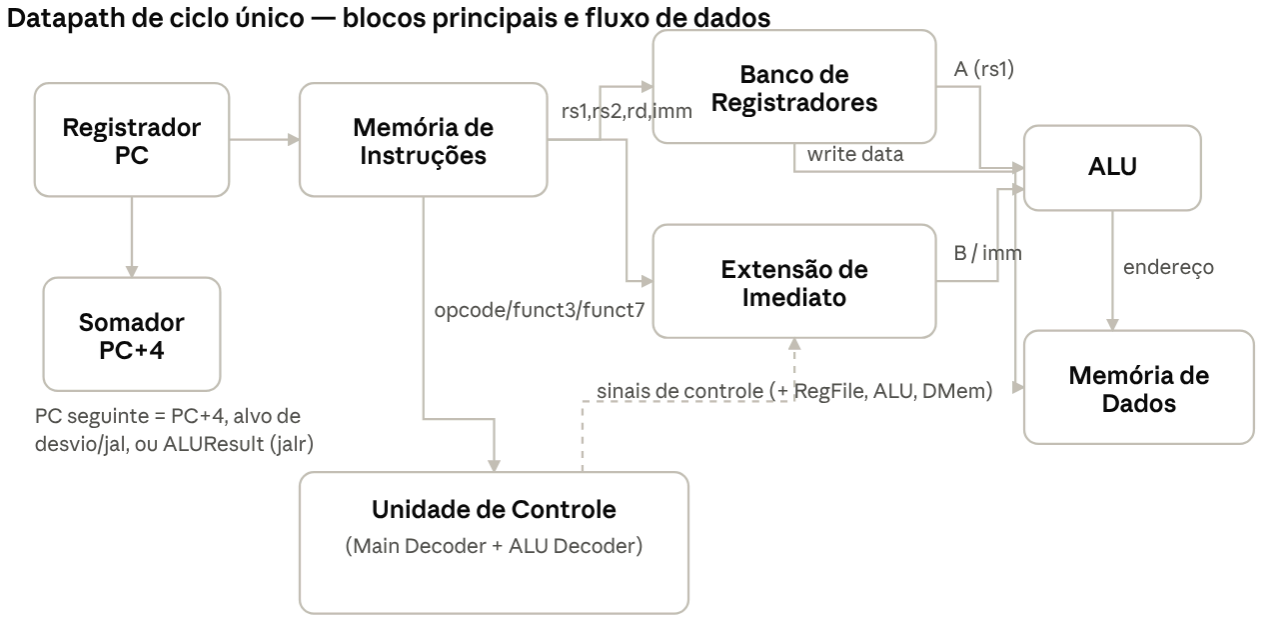
O núcleo adota a microarquitetura de **ciclo único** (single-cycle) descrita no Capítulo 7 de Harris & Harris. Nela, cada instrução — busca, decodificação, execução, acesso à memória e escrita do resultado — é completada em um único pulso de clock. Não há pipeline, nem múquinas de estado multiciclo: o caminho de dados é puramente combinacional entre um registrador de PC e o próximo, e toda a lógica de controle é gerada a partir do opcode/funct3/funct7 da instrução corrente.

**Por que essa escolha:**

- É a microarquitetura ensinada no capítulo de referência adotado pela disciplina, o que facilita a verificação cruzada com a outra equipe e com o material de aula.
- Para um ISA deliberadamente reduzido (sem necessidade de alto desempenho ou throughput), o ciclo único evita a complexidade de hazards, forwarding e stalls que um pipeline exigiria — complexidade que não traria benefício real para o objetivo do PBL (rodar corretamente o jogo Genius).
- Mantém a correspondência direta entre a estrutura do RTL e os diagramas do livro-texto (Figuras 7.8 a 7.63), o que torna a documentação e a verificação (Frente 2, por outra equipe) mais previsíveis.

A organização interna segue a divisão clássica do livro: um **Datapath** (caminho de dados) e um **Controller** (unidade de controle, dividida em Main Decoder + ALU Decoder), encapsulados no módulo de topo `riscvsingle`, que por sua vez é conectado a memórias de instrução e dados externas no wrapper de simulação `riscv_single_top` (equivalente à Figura 7.63).

# Visão geral do datapath




datapath de ciclo único · 8 blocos principais

O fluxo de uma instrução, ciclo a ciclo: o **PC** endereça a **Memória de Instruções**, que entrega a instrução de 32 bits para o **Banco de Registradores** (campos rs1/rs2/rd), para a unidade de **Extensão de Imediato** (campos de imediato, conforme o tipo) e para a **Unidade de Controle** (opcode/funct3/funct7). O operando A vem do registrador rs1; o operando B é o registrador rs2 ou o imediato estendido, escolhido pelo mux `ALUSrcB` (e, no caso de `auipc`, o próprio PC entra como operando A via `ALUSrcA`). A **ALU** calcula o resultado (aritmética/lógica, endereço de memória, ou comparação de branch). A **Memória de Dados** é acessada para `lw` / `lb` (leitura) ou `sw` / `sb` (escrita, com o dado vindo do banco de registradores). O resultado gravado de volta no banco de registradores é escolhido por um mux de 4 vias (`ResultSrc`): resultado da ALU, dado lido da memória, PC+4 (para `jal` / `jalr`) ou o imediato estendido (para `lui`). O próximo valor do PC é escolhido por um mux de 3 vias (`PCSrc`): PC+4, o alvo de desvio/salto (PC + imediato), ou o resultado da ALU (para `jalr`). Todas essas escolhas são comandadas pela Unidade de Controle, detalhada na próxima seção.

# Unidade de controle

Segue a divisão do livro-texto em dois decodificadores: o **Main Decoder** lê o opcode e gera os sinais de controle de alto nível; o **ALU Decoder** lê `ALUOp` (vindo do Main Decoder) junto com `funct3` / `funct7` e gera o código de 3 bits que comanda a ALU.

## Main Decoder (`maindec.sv`)

| Instr | op | RegWrite | ImmSrc | ALUSrcA | ALUSrcB | MemWrite | ResultSrc | Branch | ALUOp | Jump | Jalr |
|---|---|---|---|---|---|---|---|---|---|---|---|
| lw/lb | 0000011 | 1 | 000 | 0 | 1 | 0 | 01 | 0 | 00 | 0 | 0 |
| sw/sb | 0100011 | 0 | 001 | 0 | 1 | 1 | xx | 0 | 00 | 0 | 0 |
| R-type | 0110011 | 1 | xxx | 0 | 0 | 0 | 00 | 0 | 10 | 0 | 0 |
| I-type ALU | 0010011 | 1 | 000 | 0 | 1 | 0 | 00 | 0 | 10 | 0 | 0 |
| branches | 1100011 | 0 | 010 | 0 | 0 | 0 | xx | 1 | 01 | 0 | 0 |
| jal | 1101111 | 1 | 011 | x | x | 0 | 10 | 0 | xx | 1 | 0 |
| jalr | 1100111 | 1 | 000 | 0 | 1 | 0 | 10 | 0 | 00 | 1 | 1 |
| lui | 0110111 | 1 | 100 | x | x | 0 | 11 | 0 | xx | 0 | 0 |
| auipc | 0010111 | 1 | 100 | 1 | 1 | 0 | 00 | 0 | 00 | 0 | 0 |
| ecall | 1110011 | 0 | xxx | x | x | 0 | xx | 0 | xx | 0 | 0 |

`ALUSrcA` seleciona entre o registrador rs1 (0) e o PC (1, usado por `auipc`). `ALUSrcB` seleciona entre rs2 (0) e o imediato estendido (1). `ResultSrc` (2 bits) escolhe o que é gravado de volta no banco de registradores: 00 = resultado da ALU, 01 = dado lido da memória, 10 = PC+4, 11 = imediato estendido (`lui`). `Branch` e `Jump` / `Jalr` controlam o mux `PCSrc` de 3 vias descrito na seção anterior.

## ALU Decoder (`aludec.sv`)

| ALUOp | funct3 | funct7[5] | opcode | Operação | ALUControl |
|---|---|---|---|---|---|
| 00 | — | — | lw/sw/lb/sb/jalr/auipc | add | 000 |
| 01 | 000 (beq/bne) | — | branch | sub (comparação por igualdade) | 001 |
| 01 | 101/110 (bge/bltu) | — | branch | sub (usa flags lt/ltu da ALU) | 001 |
| 10 | 000 | 0 (ou I-type) | R/I-type | add | 000 |
| 10 | 000 | 1 | R-type | sub | 001 |
| 10 | 111 | — | R/I-type | and | 010 |
| 10 | 110 | — | R/I-type | or | 011 |
| 10 | 100 | — | R/I-type | xor | 100 |
| 10 | 001 | — | R/I-type | sll | 101 |
| 10 | 101 | — | R/I-type | srl | 110 |

Para os branches, o `controller.sv` combina o `funct3` da instrução com as flags `zero`, `lt` (signed) e `ltu` (unsigned) produzidas pela ALU para decidir `branch_taken`: `beq` →zero, `bne` →!zero, `bge` →lt==0, `bltu` →ltu==1. O sinal final `pcsrc` (2 bits) é 00

(PC+4) quando nenhum desvio é tomado, 01 (PCTarget) quando `Branch & branch_taken` ou `Jump`, e 10 (ALUResult) quando `Jalr`. O controlador também expõe um sinal `ecall`, levantado quando o opcode decodificado é `1110011`, usado como marcador de fim de execução na validação (Seção 7).

## Extensões feitas além do livro-base

O Capítulo 7 de Harris & Harris constrói, como exemplo didático, um núcleo que cobre `add/sub/and/or/slt`, `lw/sw`, `beq`, e apresenta `addi` / `jal` como exercício de ampliação ("More Instructions"). Esse subconjunto não é suficiente para rodar o programa do PBL02, então o datapath e o decodificador foram estendidos:

| Extensão | Por quê | O que mudou no hardware |
|---|---|---|
| `lb` / `sb` (byte) | O jogo manipula caracteres/strings byte a byte, não só palavras de 32 bits | `dmem.sv` ganhou seleção de largura por `funct3`, com sign-extend no `lb` |
| `bne` / `bge` / `bltu` | O livro só ensina `beq`; o jogo precisa de comparações de diferença, maior/igual (signed) e menor (unsigned) para loops e lógica de pontuação | A ALU ganhou flags `lt` (signed) e `ltu` (unsigned), calculadas por comparação direta em SystemVerilog (não derivadas da subtração); `controller.sv` combina `funct3` + flags para decidir `branch_taken` |
| `jalr` | Chamadas/retornos de sub-rotina indiretos (ex.: retorno de função via endereço em registrador) | `PCSrc` deixou de ser mux de 2 vias (PC+4 / alvo de desvio) e passou a 3 vias, incluindo o `ALUResult` |
| `lui` | Carregar constantes/endereços de 32 bits — o imediato de 12 bits de `addi` não alcança os endereços usados pelo jogo (ex.: base de um vetor na memória de dados) | `extend.sv` ganhou o formato de imediato tipo U; `ResultSrc` passou de 2 para 4 opções (incluindo o imediato estendido direto) |
| `auipc` | Complementa `lui` para construir endereços relativos ao PC (usado tipicamente em pseudo-instruções como `la`) | `ALUSrcA` foi adicionado como mux (registrador rs1 vs. PC) na entrada da ALU |

| Extensão | Por quê | O que mudou no hardware |
|---|---|---|
| `ecall` | Não faz parte do jogo em si; adicionada como ponto de parada controlável | Um sinal `ECall` dedicado, levantado pelo controlador e observado pelo testbench para finalizar a simulação de forma determinística |

Em resumo: o núcleo manteve a filosofia de ciclo único e a estrutura Main/ALU Decoder do livro, mas ampliou o repositório de formatos de imediato (incluindo o tipo U), o número de vias dos muxes `ALUSrcA` / `ResultSrc` / `PCSrc`, e as flags de comparação da ALU — o mínimo necessário para cobrir exatamente as instruções da Seção 2, sem implementar o RV32I completo.

## Validação

Antes de integrar o núcleo ao ambiente UVM da Frente 2 (que será construído pela equipe parceira para verificar este processador), o RTL foi validado por um **sanity-check** dedicado — um testbench simples, não-UVM ( `tb/testbench.sv` ), que instancia o SoC de simulação ( `riscv_single_top` = `riscvsingle` + `imem` + `dmem` ), gera clock de 10 ns e aplica reset, e imprime PC/instrução a cada ciclo e cada escrita em memória.

**Programa de teste** ( `sim/encode.py` , montado à mão em Python, já que não havia toolchain RISC-V disponível no ambiente de desenvolvimento): exercita deliberadamente todas as classes de instrução suportadas — carga de constante grande ( `lui` ), aritmética ( `addi` , `add` ), armazenamento e leitura de palavra ( `sw` / `lw` ), armazenamento e leitura de byte ( `sb` / `lb` ), um loop controlado por comparação unsigned ( `bltu` ), e `auipc` para capturar o PC corrente — terminando em `ecall` .

Ao atingir o `ecall` , o testbench compara os registradores-resultado com os valores esperados:

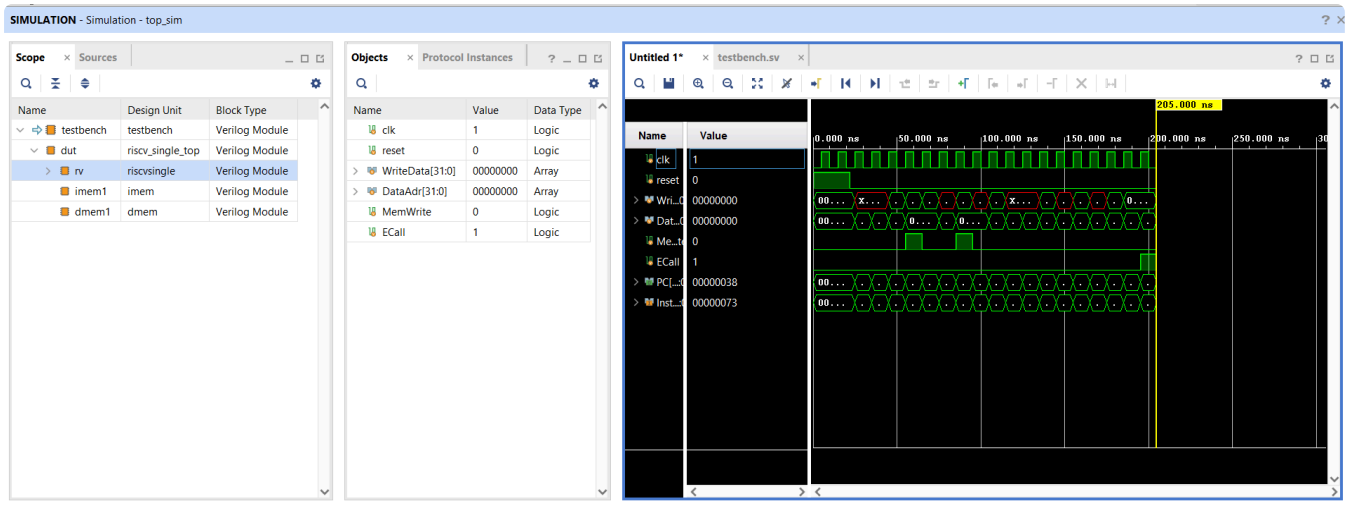
| Registrador | Papel no teste | Esperado | Obtido |
|---|---|---|---|
| x28 (t3) | lido de volta da memória ( `sw` + `lw` ) | 15 | 15 |
| x30 (t5) | lido de volta da memória ( `sb` + `lb` ) | 65 | 65 |
| x12 (a2) | `auipc` no endereço 0x34 | 0x34 | 0x34 |
| x10 (a0) | contador do loop ( `bltu` ) | 3 | 3 |

**Metodologia em duas etapas:**

1. **Validação local** com Icarus Verilog ( `iverilog -g2012` ), usada como checagem rápida de sintaxe/funcionalidade durante o desenvolvimento do RTL.
2. **Validação no fluxo oficial da disciplina** — Vivado, via `verif-env-template` (CMake + `xvlog` / `xelab` / `xsim` ) — rodada diretamente na máquina da autora (Windows + Git Bash), incluindo todo o processo de configuração do ambiente (CMake, Make do próprio Vivado, gerador "Unix Makefiles", execução do `xsim` via `cmd //c` para contornar uma incompatibilidade do wrapper `.bat` do Xilinx com o Git Bash/MINGW64).

**Resultado:** a simulação no Vivado XSim rodou até o `$finish` e reproduziu exatamente os mesmos valores da validação local — todos os quatro registradores de checagem bateram com o esperado, confirmando que o núcleo funciona corretamente no toolchain real exigido pela disciplina, não apenas em simulação auxiliar.

**Evidências da simulação no Vivado (XSim):**




Formas de onda do datapath (clock, PC, instrução corrente e sinais de memória) durante a execução completa do programa, vistas no Vivado Simulator (modo gráfico, `xsim -gui` ).

```
---- ECALL em PC=00000038 ----
a0 = 3 (0x00000003)
t3 (mem[0x2000]) esperado=15 -> x28 = 15
t5 (mem[0x2004]) esperado=65 -> x30 = 65
a2 (auipc)       esperado=0x34 -> x12 = 00000034
$finish called at time : 205 ns : File "C:/Users/marin/Documents/pbl03/tb/testbench.sv" Line 44
```

Saída do console no momento do `ecall` (PC=0x00000038): os quatro registradores de checagem bateram exatamente com o esperado ( `t3=15` , `t5=65` , `a2=0x34` , `a0=3` ), e a simulação terminou normalmente ( `$finish` em 205 ns) — confirmando, no toolchain oficial da disciplina, o mesmo resultado já validado localmente.

# Genius no processador

O programa Genius do PBL02, em assembly RISC-V, roda no processador **sem nenhuma alteração no código**: nos 5 cenários simulados no Vivado xsim 2024.1 a pontuação final foi a esperada e não houve divergência na sequência de cores ( `5 cenarios passaram, 0 falharam` ). A verificação é feita só em simulação; os periféricos do jogo são modelos em RTL, não hardware da Basys3.

## Como foi montado

O jogo precisa de LEDs, chaves, botões e um relógio, que o núcleo não tem. Um SoC de teste ( `genius_soc.sv` ) liga o núcleo às memórias e a um bloco de periféricos mapeados em memória ( `genius_periph.sv` ). O assembly não muda: os endereços dos periféricos entram na montagem como símbolos ( `--defsym` ), e um pequeno `crt0.s` inicializa o `sp` , como o Ripes faz.

| Endereço | O que é | Acesso |
|---|---|---|
| 0x0000_0000 | Memória de instruções (161 palavras, `genius.hex` ) | leitura |
| 0x0000_2000 | Dados: `seq_buffer` em 0x2000, `cores` em 0x2020 (48 bytes, `data.hex` ) | lb/sb |
| 0x0000_3FF0 | Topo da pilha ( `sp` , definido pelo `crt0.s` ) | lw/sw |
| 0xF000_0000 | LEDs: 4 LEDs de 24 bits (UP, DOWN, LEFT, RIGHT) | escrita |
| 0xF000_0100 | Chaves: bit 4 início, bit 3 modo, bit 2 velocidade, bits 1:0 dificuldade | leitura |
| 0xF000_0200 | D-Pad: 4 botões com trava, limpa quando o programa lê | leitura |
| 0xF000_0300 | Relógio (RTC): avança 1 ms por ciclo de clock, para os atrasos de 2 s custarem ~2000 ciclos | leitura |

O desassembly confirmou que todas as instruções do Genius já estavam na ISA implementada ( `lui` , `auipc` , `lb` , `sb` , `bne` , `bge` , `bltu` , `jalr` , `ecall` entre elas), então o núcleo não precisou de nenhuma mudança. A montagem usa `riscv64-unknown-elf-as -march=rv32i` , `ld` com `.text` em 0x0 e `.data` em 0x2000, e `objcopy` para gerar o `genius.hex` .

## Testbench

O `tb/genius_tb.sv` joga o jogo sozinho: aperta os botões do D-Pad, observa os LEDs e compara cada cor mostrada com um modelo de referência (LFSR de 8 bits, semente 0xA5, taps nos bits 7, 5, 4 e 3). Também confere a pontuação impressa no `ecall` e o maior tamanho de sequência (registrador `s3`).

| # | Modo | Dificuldade | Velocidade | Situação | Pontuação | `s3` |
|---|---|---|---|---|---|---|
| 1 | Siga | Fácil | Lento | Partida completa, 8 rodadas | 36 | 8 |
| 2 | Siga | Fácil | Rápido | Erro de propósito na rodada 3 | 6 | 8 |
| 3 | Siga | Médio | Rápido | Partida completa, 16 rodadas | 544 | 16 |
| 4 | Siga | Difícil | Lento | Erro de propósito na rodada 2 | 3 | 32 |
| 5 | Mando eu | Fácil | Lento | UP, RIGHT, DOWN; erro na rodada 3 | 3 | 8 |

Nos cinco cenários, a pontuação obtida foi igual à esperada, o `s3` bateu e as divergências foram 0.

## Como rodar

```
source setup_env.sh
cmake -S . -B build -G "Unix Makefiles" -DTOP_NAME=genius_tb -
DTEST_NAME=sanity -DSEED=1 -DVIVADO_PARMS="--R"
cp genius.hex data.hex build/
make SHELL="$(which bash)" compile
make SHELL="$(which bash)" sim
cd build
cmd //c "xsim top_sim -R"
```

O último `make` termina com o erro `Unsupported architecture: i686-pc` do Git Bash; a elaboração já foi concluída e o `xsim` é chamado à parte, por `cmd`.

## Evidências

**Execução completa no `xsim`.** Os cinco cenários, cada um com `PASSOU`, pontuação igual à esperada e 0 divergências.

```
****** xsim v2024.1 (64-bit)
  **** SW Build 5076996 on Wed May 22 18:37:14 MDT 2024
  **** IP Build 5075265 on Wed May 22 21:45:21 MDT 2024
  **** SharedData Build 5076995 on Wed May 22 18:29:18 MDT 2024
  **** Start of session at: Tue Oct 13 08:56:35 2026
    ** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
    ** Copyright 2022-2024 Advanced Micro Devices, Inc. All Rights Reserved.

source xsim.dir/top_sim/xsim_script.tcl
# xsim {top_sim} -autoloadwcfg -runall
Time resolution is 1 ps
run -all
Genius no nucleo RISC-V (PBL03) - 161 palavras de programa carregadas

=== Cenario 1: modo Siga, facil, lento, partida completa (8 rodadas) ===
  pontuacao = 36 (esperado 36) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
  >>> Cenario 1: modo Siga, facil, lento, partida completa (8 rodadas): PASSOU

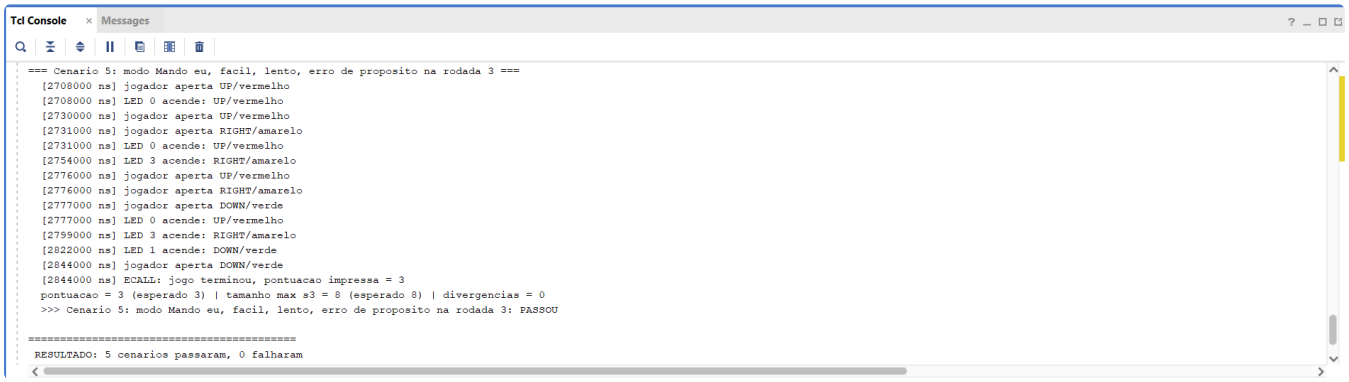
=== Cenario 2: modo Siga, facil, rapido, erro de proposito na rodada 3 ===
  [821000 ns] LED 2 acende: LEFT/azul
  [834000 ns] jogador aperta LEFT/azul
  [834000 ns] LED 2 acende: LEFT/azul
  [847000 ns] LED 1 acende: DOWN/verde
  [859000 ns] jogador aperta LEFT/azul
  [860000 ns] jogador aperta DOWN/verde
  [860000 ns] LED 2 acende: LEFT/azul
  [873000 ns] LED 1 acende: DOWN/verde
  [885000 ns] LED 2 acende: LEFT/azul
  [897000 ns] jogador aperta RIGHT/amarelo
  [898000 ns] ECALL: jogo terminou, pontuacao impressa = 6
  pontuacao = 6 (esperado 6) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
  >>> Cenario 2: modo Siga, facil, rapido, erro de proposito na rodada 3: PASSOU

=== Cenario 3: modo Siga, medio, rapido, partida completa (16 rodadas) ===
  pontuacao = 544 (esperado 544) | tamanho max s3 = 16 (esperado 16) | divergencias = 0
  >>> Cenario 3: modo Siga, medio, rapido, partida completa (16 rodadas): PASSOU

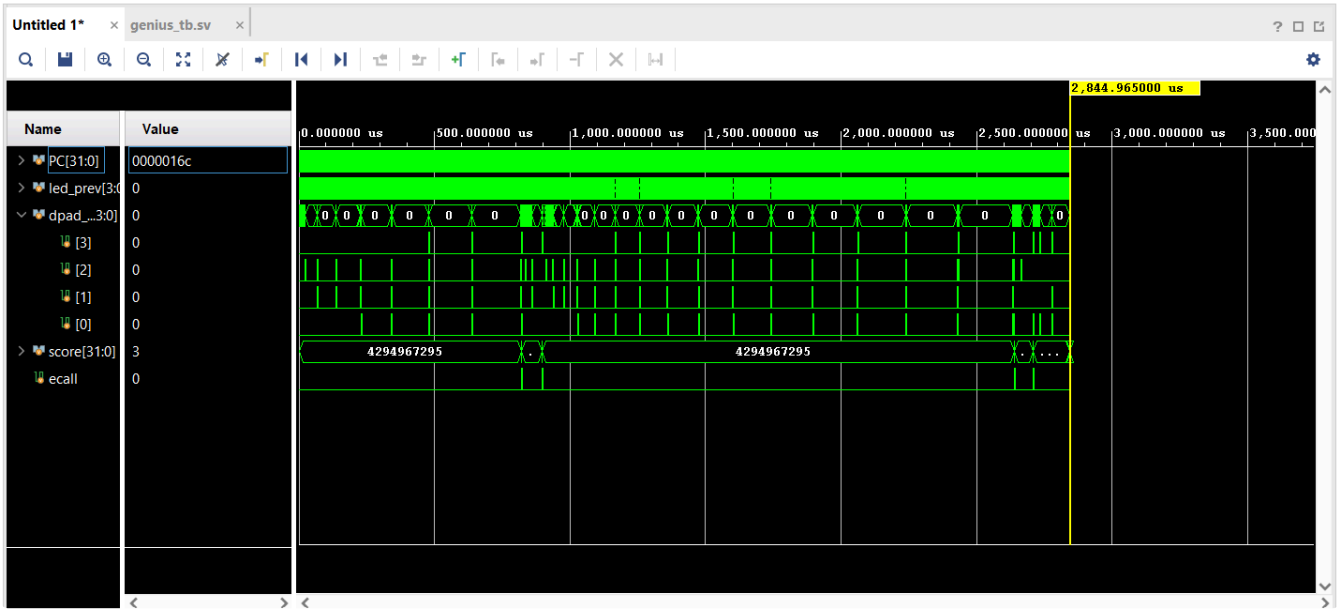
=== Cenario 4: modo Siga, dificil, lento, erro de proposito na rodada 2 ===
  pontuacao = 3 (esperado 3) | tamanho max s3 = 32 (esperado 32) | divergencias = 0
  >>> Cenario 4: modo Siga, dificil, lento, erro de proposito na rodada 2: PASSOU

=== Cenario 5: modo Mando eu, facil, lento, erro de proposito na rodada 3 ===
  [2708000 ns] jogador aperta UP/vermelho
  [2708000 ns] LED 0 acende: UP/vermelho
  [2730000 ns] jogador aperta UP/vermelho
  [2731000 ns] jogador aperta RIGHT/amarelo
  [2731000 ns] LED 0 acende: UP/vermelho
  [2754000 ns] LED 3 acende: RIGHT/amarelo
  [2776000 ns] jogador aperta UP/vermelho
  [2776000 ns] jogador aperta RIGHT/amarelo
  [2777000 ns] jogador aperta DOWN/verde
  [2777000 ns] LED 0 acende: UP/vermelho
  [2799000 ns] LED 3 acende: RIGHT/amarelo
  [2822000 ns] LED 1 acende: DOWN/verde
  [2844000 ns] jogador aperta DOWN/verde
  [2844000 ns] ECALL: jogo terminou, pontuacao impressa = 3
  pontuacao = 3 (esperado 3) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
```

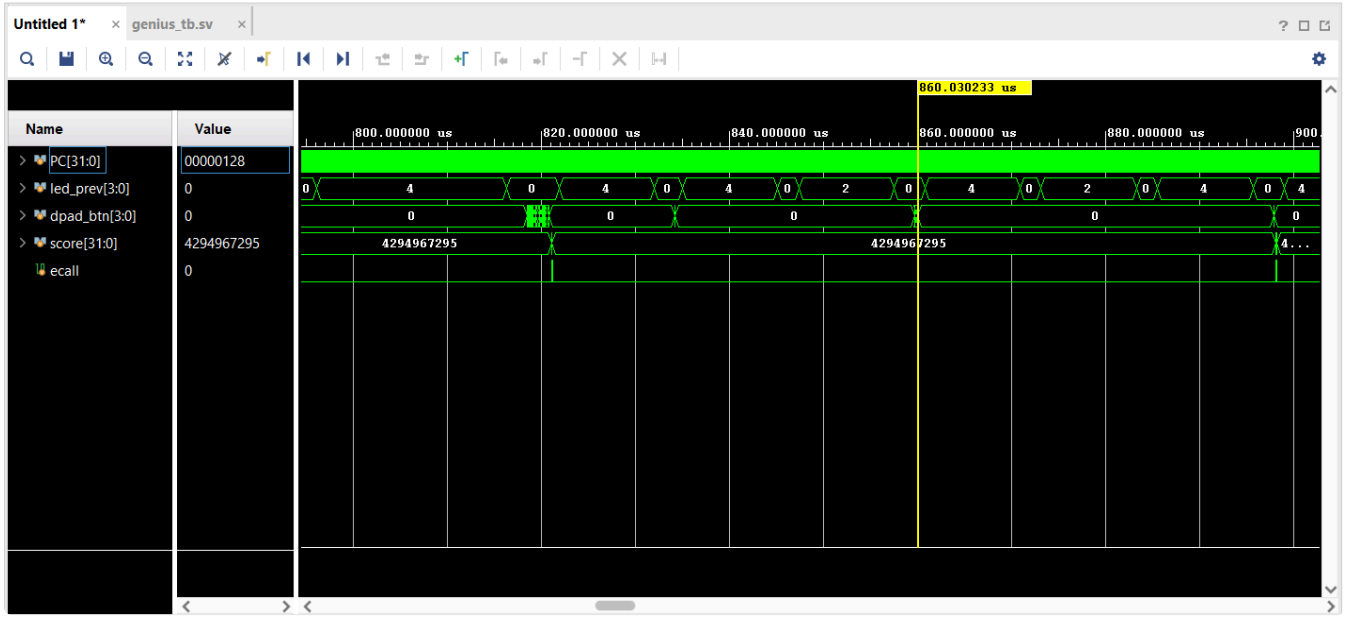
**Resultado final no console do Vivado.** Pontuação 3 (esperado 3) no modo Mando eu e a linha `RESULTADO: 5 cenarios passaram, 0 falharam`.



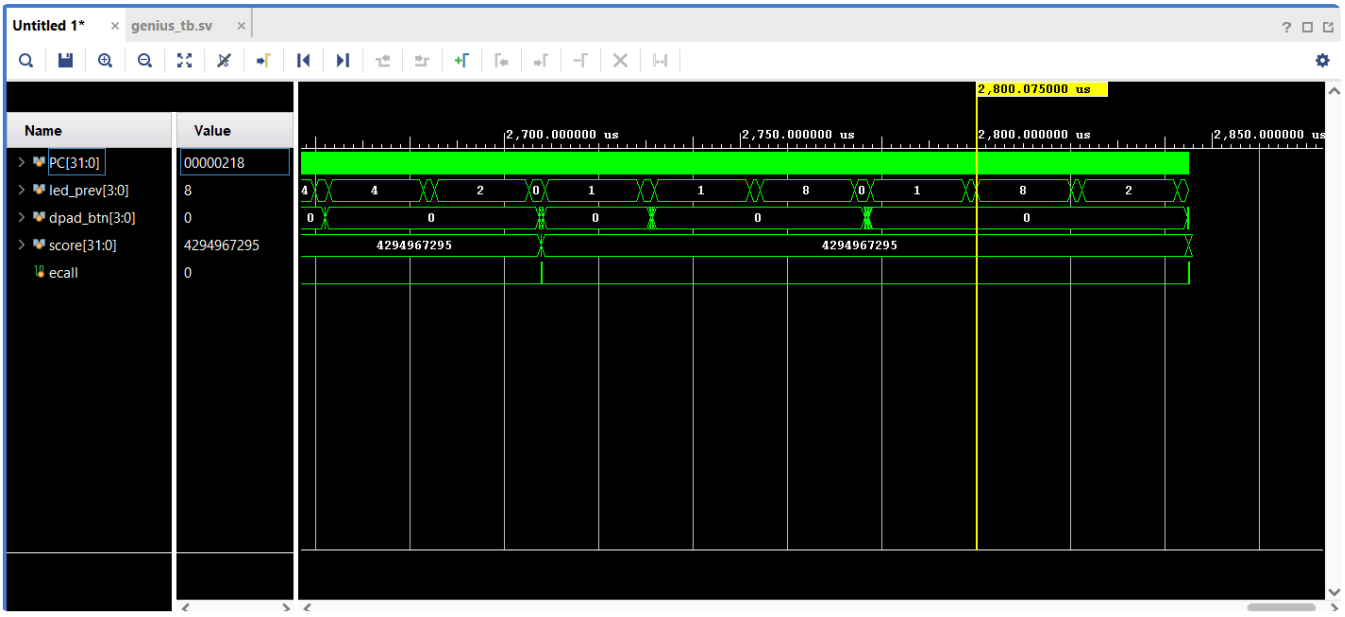

**Visão geral das ondas (0 a 2.845 µs).** Os pulsos de `ecall` marcam o fim de cada partida e o `score` mostra 4294967295 (−1) enquanto a partida está em andamento.



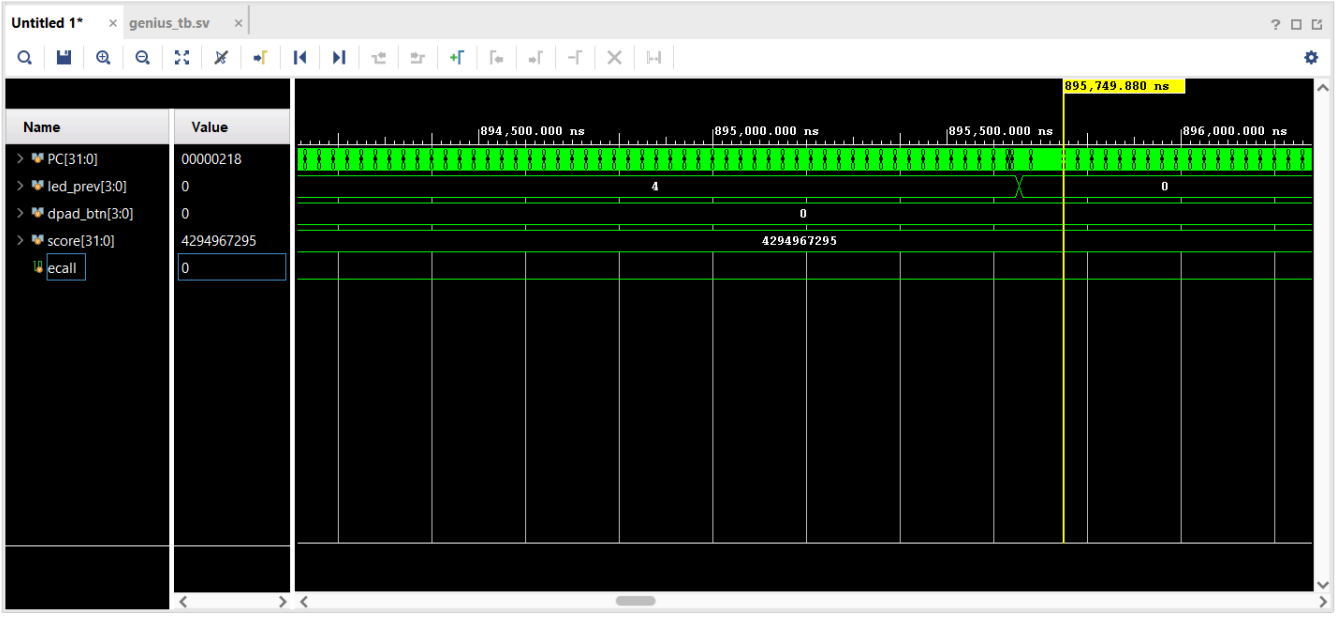

**Modo Siga (cenário 2, 800 a 900 µs).** O jogo mostra a sequência crescendo no `led_prev` (4; 4, 2; 4, 2, 4), com o LED apagando entre as cores. O jogador erra de propósito na rodada 3 e a partida termina com pontuação 6.




**Modo Mando eu (cenário 5, 2.700 a 2.850 µs).** O jogador escolhe UP, RIGHT e DOWN; o jogo repete a sequência acumulada (1; 1, 8; 1, 8, 2) e a partida termina na rodada 3 com pontuação 3. O pulso de `ecall` em ~2.708 µs marca o fim do cenário 4.




**Processador em execução (escala de ns, ~895 µs).** Com zoom em nanossegundos, o `PC` muda a cada ciclo de clock, mostrando o núcleo executando as instruções do Genius.




# Próximos passos

Com a Frente 1 (projeto RTL) validada no fluxo oficial, o trabalho que falta para o PBL03 é a **Frente 2 — ambiente de verificação UVM**: diferente do testbench de sanity-check usado aqui, essa frente consiste em construir, seguindo o template `verif-env-template`, um ambiente UVM completo (agentes, modelo de referência, scoreboard, cobertura funcional e testes) para **verificar o processador da equipe parceira** — não este núcleo.

Pendente:

- Obter da equipe parceira a especificação/ISA do processador dela a ser verificado.

- Estruturar o ambiente UVM no diretório `tb/` (agentes, modelo de referência, scoreboard, cobertura) seguindo a convenção do template.
- Definir o plano de testes/cobertura que cubra o ISA limitado do processador parceiro.
- Preparar a demonstração de 10 minutos e o relatório final, reunindo os resultados das duas frentes (este núcleo + o ambiente que verificará o núcleo da outra equipe).